

GMRES算法DCU优化技术总结报告

一、项目总览

优化工作概述

我们针对稀疏线性方程组求解的GMRES算法，在DCU平台上进行了系统性的性能优化工作。通过从基础移植到深度优化的完整技术演进。最终在严格保证计算精度的前提下，实现了显著的性能提升。

最终性能成果

实现 98.75% 通过率，通过案例残差均满足 $1e-6$ 要求, 加速比在20-40倍不等。

二、技术路线演进

我们的优化工作经历了从基础移植到深度优化的技术演进过程。

技术选型：

- 基础移植dcu
- 直接调用hipBLAS和hipSPARSE库函数，建立双精度基准版本
- 启用warmup,givens旋转从cpu转移到gpu
- 混合精度优化
- 手动内核开发

决策依据：

- 库函数起步：基于hipBLAS和hipSPARSE库函数经过深度优化，能够快速建立性能基准并验证算法正确性。
- 启用warmup可降低函数固定开销,givens旋转是数据密集型操作，可利用GPU的并行计算能力提升性能。
- 混合精度：单精度计算可提供2倍内存带宽利用率和近似2倍计算吞吐量，但关键精度敏感操作仍需保持双精度。
- 自主内核开发：库函数的固定调用开销限制了性能上限，手动实现内核组件能够针对算法特性进行深度优化。

三、核心优化手段与性能分析

1.线性算子的DCU移植

方法说明：

我们利用竞赛允许的 hipBLAS 和 hipSPARSE 库，将原始代码中手写的循环替换为高效的库函数调用。成功将所有计算密集型的线性代数操作从CPU迁移到DCU上执行。

代码实现：

```
1 // 典型示例: CSR-Vector SpMV + AXPY + ||r||^2 分块归约 (DCU/HIP)
2 template<typename IndexType, unsigned VECTORS_PER_BLOCK, unsigned
  THREADS_PER_VECTOR>
3 __global__ void spmv_vector_minus_b_partial(
```

```

4      ...
5  ){
6      const IndexType tid = VECTORS_PER_BLOCK * THREADS_PER_VECTOR *
      blockIdx.x + threadIdx.x;
7      ...
8      __syncthreads();
9      if (threadIdx.x == 0) {
10         double blk = 0.0;
11         #pragma unroll
12         for (unsigned v = 0; v < VECTORS_PER_BLOCK; ++v) blk += s_r2[v];
13         partial[blockIdx.x] = blk; // 写出本block的部分和
14     }
15 }

```

性能收益:

- 线性算子迁移至DCU并融合内核，带宽利用与并行度显著提升，启动与通信开销下降，mtx1矩阵运行时间从3291ms减少至724ms。

```

=====
mtx1: M = 503625, N = 503625
start running gmres
iters = 107, time = 724.216ms, resid = 9.91841e-07
HIP_PROF:on Sun Nov  9 13:58:01 2025
HIP_PROF:process id '112197'
HIP_PROF:db file './hip-prof-112197.db'
HIP_PROF:out put file 'result_112197'
HIP_PROF:profiling './gmres /work/home/xdzs2025_bhz/GMRES/sparseMatrixSet/mtx1.bin'
HIP_PROF:process begin
HIP_PROF:process end run total cost:8(s)

```

性能瓶颈: 模块加载 (hipModuleLoad)

HIP PROF:hip API statistics				
Name	Calls	TotalDurationNs	AverageNs	Percentage
hipModuleLoad	2	5397425334	2698712667	85.2693664407609

2.全面的GPU预热

方法说明:

为确保计时结果的准确性和稳定性，必须排除DCU首次执行任务时产生的“冷启动”开销。我们在计时开始前设计了全面的预热流程。

代码实现:

```

1  // ===== warmup 开始 =====
2  {
3      // 1. 预热数据类型转换kernel
4      cast_d2f<<<grid_for_vec(nnz), TPB>>>(nnz, d_vals, d_vals_f);
5
6      // 2. 预热SpMV kernel (使用实际运行时的配置)
7      constexpr unsigned V = 4, TV = 16;
8      dim3 block(V * TV), grid((N + V - 1) / V);
9      spmv_vector_minus_b_partial<uint, V, TV><<<grid, block>>>(
10         N, d_rowPtr, d_colInd, d_vals, d_x, d_b, d_r, d_partial_spmv);
11
12      // 3. 预热单精度SpMV
13      spmv_vector_f32<uint, V, TV><<<grid, block>>>(

```

```

14         N, d_rowPtr, d_colInd, d_vals_f, d_vf, d_rf);
15
16         // 4. 确保所有预热操作完成
17         HIP_CHECK(hipDeviceSynchronize());
18     }
19     // ===== warmup 结束 =====

```

性能收益:

- 全面预热消除冷启动/JIT开销，避免首轮抖动；计时更准、更稳且可复现，端到端时延显著下降，加载从85%降至72%，mtx1矩阵运行时间减少至330ms。

```

=====
Starting Performance Profiling for /work/home/xdzs2025_bhz/GMRES/sparseMatrixSet/mtx1.bin...
=====
mtx1: M = 503625, N = 503625
start running gmres
iters = 80, time = 330.234ms, resid = 9.92996e-07
HIP_PROF:on Sun Nov  9 12:27:41 2025
HIP_PROF:process id '5387'
HIP_PROF:db file './hip-prof-5387.db'
HIP_PROF:out put file 'result_5387'
HIP_PROF:profiling './gmres /work/home/xdzs2025_bhz/GMRES/sparseMatrixSet/mtx1.bin'
HIP_PROF:process begin
HIP_PROF:process end run total cost:7(s)

```

HIP PROF:hip API statistics				
Name	Calls	TotalDurationNs	AverageNs	Percentage
hipModuleLoad	2	2035278142	1017639071	72.8668883407929

3.混合精度设计

方法说明:

- 外层残差计算：全程使用双精度，确保收敛判断的数值准确性
- 内层Krylov构建：采用混合精度计算，在保证数值稳定性的前提下提升计算效率
- 点积运算优化：单精度数据读取配合双精度累加，有效平衡内存带宽与计算精度需求

代码实现:

```

1 // 批量点积优化实现：单精度读取 + 双精度累加
2 __global__ void dot_multi_partial_f32acc_f64(
3     int n, int m, const float* w, const float* v,
4     int ldv, double* partial)
5 {
6     int tid = threadIdx.x;
7     int gid = blockIdx.x * blockDim.x + tid;
8     // 每个线程维护m个双精度累加器
9     double acc[MAX_M] = {0};
10    // 单精度读取，双精度累加
11    for (int i = gid; i < n; ...) {
12        double wi = (double)w[i]; // 单精度→双精度
13        for (int j = 0; j < m; ++j) {
14            acc[j] += wi * (double)v[...];
15        }
16    }
17    // Block内规约
18    // 二分规约...
19 }

```

```

20
21 // ===== 初始版本: 全双精度 =====
22 double *v_d;
23 hipMalloc(&v_d, (RESTART_TIMES + 1) * N * sizeof(double)); // 168N bytes
24
25 // ===== 优化版本: Krylov空间用单精度 =====
26 float *d_vf; // 单精度Krylov子空间
27 double *d_r; // 双精度残差向量
28 hipMalloc(&d_vf, (m + 1) * N * sizeof(float)); // 84N bytes (节省50%)
29 hipMalloc(&d_r, N * sizeof(double)); // 8N bytes

```

性能收益:

- 混合精度将A与Krylov向量用float、关键累加用double，内存占用减半、带宽效率与吞吐近翻倍，在保持1e-6精度下显著提速。mtx1矩阵运行时间减少至206ms。

```

mtx1: M = 503625, N = 503625
start running gres
iters = 120, time = 206.159ms, resid = 6.46418e-07
HIP_PROF:on Sun Nov 9 06:04:32 2025
HIP_PROF:process id '124676'
HIP_PROF:db file './hip-prof-124676.db'
HIP_PROF:out put file 'result_124676'
HIP_PROF:profiling './gmres /work/home/xdzs2025_bhz/GMRES/sparseMatrixSet/mtx1.bin'
HIP_PROF:process begin
HIP_PROF:process end run total cost:8(s)

```

4.内核优化

方法说明:

- 手动实现spmv内核,并进行向量化处理,融合操作以及自适应优化:** 采用多线程合作处理单行矩阵的向量化计算模式,提升计算单元利用率,将稀疏矩阵向量乘法与向量线性组合操作融合为单一内核,减少内核调用开销和数据传输耗时,基于矩阵稀疏特征动态调整线程配置;
- 将原本在CPU端顺序执行的Givens旋转完全移植到DCU设备端执行**实现批处理**旋转内核,单次内核调用完成所有内迭代的Givens旋转计算,消除主机-设备间的旋转参数数据传输,减少同步开销;
- 混合精度点积累加内核,** 单精度读取提升带宽效率,双精度累加保证计算精度实现一次性批量计算多个向量点积,大幅减少规约操作的总开销

代码实现:

```

1 // ===== 初始版本: 3个独立kernel =====
2 spmv_kernel<<<...>>>(A, x, r); // r = A*x
3 daxpy_kernel<<<...>>>(-1.0, b, r); // r = r - b
4 double norm = gpu_norm(r, N); // norm = ||r||
5
6 // ===== 优化版本: 融合为单个kernel =====
7 template<typename IndexType, unsigned VECTORS_PER_BLOCK, unsigned
  THREADS_PER_VECTOR>
8 __global__ void spmv_vector_minus_b_partial(
9     const IndexType row_num,
10    const IndexType *__restrict__ rowPtr,
11    const IndexType *__restrict__ colInd,
12    const double *__restrict__ vals,
13    const double *__restrict__ x,
14    const double *__restrict__ b,

```

```

15     double *__restrict__ r,           // 输出残差向量
16     double *__restrict__ partial)    // 输出||r||^2的部分和
17 {
18     // ... 线程索引计算 ...
19     // 1. SpMV: sum = A[row,:] * x
20     double sum = 0.0;
21     for (IndexType jj = ...; jj < ...; ...) {
22         sum += vals[jj] * x[colInd[jj]];
23     }
24     // warp内规约...
25     // 2. AXPY + 范数平方: ri = sum - b[row], r2 = ri^2
26     // 3. Block级规约: 合并平方和
27 }

```

- spmv内核采用CSR-Vector模板实现自适应优化,增加网格大小,缩减线程块大小,每个 warp 可以同时负责矩阵多行的计算

```

1 // 根据矩阵稀疏特征选择最优配置
2 uint avg_nnz_per_row = std::max(1u, nnz / N);
3 unsigned v = 4;    // VECTORS_PER_BLOCK (固定)
4 unsigned TV;       // THREADS_PER_VECTOR (自适应)
5 if (avg_nnz_per_row <= 16)    TV = 4;    // 极稀疏: 少量线程
6 else if (avg_nnz_per_row <= 32) TV = 8;    // 稀疏
7 else if (avg_nnz_per_row <= 64) TV = 16;   // 中等稀疏
8 else                        TV = 32;    // 密集: 更多线程协作

```

性能收益:

- 手写SpMV/批量点积、算子融合、warp归约与TV自适应,减少launch与全局访存,提升占用率和带宽效率;点积100ms降至30ms,端到端显著提速。mtx1矩阵运行时间减少至144ms。

```

mtx1: M = 503625, N = 503625
start running gmres
iters = 120, time = 144.147ms, resid = 6.48257e-07
HIP_PROF:on Sun Nov  9 13:54:45 2025
HIP_PROF:process id '1110'
HIP_PROF:db file './hip-prof-1110.db'
HIP_PROF:out put file 'result_1110'
HIP_PROF:profiling './gmres /work/home/xdzs2025_bhz/GMRES/sparseMatrixSet/mtx1.bin'
HIP_PROF:process begin
HIP_PROF:process end run total cost:8(s)

```

四、总结

本项目针对GMRES算法在DCU平台上进行了系统性能优化,通过四个递进式优化阶段实现了显著的性能提升:

技术演进路径:

1. **基础移植阶段:** 利用hipBLAS/hipSPARSE库函数完成DCU移植,建立性能基线 (3291ms→724ms)
2. **预热优化阶段:** 全面GPU预热消除冷启动开销,稳定计时性能 (724ms→330ms)
3. **混合精度阶段:** 单精度Krylov空间+双精度关键累加,内存占用减半、带宽效率翻倍 (330ms→206ms)
4. **深度内核优化:** 手写SpMV融合内核、批量Givens旋转、自适应线程配置 (206ms→144ms)

核心成果：

- **性能提升：**典型案例mtx1从3291ms优化至144ms，总体加速比达20-40倍
- **精度保障：**98.75%通过率，所有通过案例残差满足 $1e-6$ 精度要求
- **技术深度：**从算法映射到内存优化，从计算精度到并行策略的全方位优化

经验总结：

本项目证明了"库函数起步→混合精度优化→手动内核深化"的优化路径在异构计算平台上的有效性。通过对计算特征的深入分析和针对性优化，在保证数值精度的前提下充分释放了DCU硬件的计算潜力，为稀疏线性求解器的异构加速提供了完整的工程实践参考。